

Interface_Business_Specification

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
 - 1.1 Purpose
 - 1.2 Scope
2. Interface Design Principles
 - 2.1 General Principles
 - 2.2 Protocol Selection
3. RESTful API Standards
 - 3.1 URL Naming Convention
 - 3.2 HTTP Method Usage
 - 3.3 Request & Response Standards
 - 3.4 Error Code Standards
 - 3.5 API Version Management
4. gRPC Standards
 - 4.1 .proto File Standards
 - 4.2 Conventions
5. Message Queue Standards
 - 5.1 Message Structure
 - 5.2 Topic Naming
 - 5.3 Message Reliability
6. Interface Documentation Requirements
 - 6.1 Mandatory Content
 - 6.2 Documentation Tools
 - 6.3 Field Definition Table Template
7. Interface Review
 - 7.1 Review Checklist
 - 7.2 Security Requirements
8. Appendix
 - 8.1 Reference Documents
 - 8.2 Templates

Interface Business Specification Standard

> Reference: Alibaba API Design Standards, Tencent Microservice Interface Standards, Google API Design Guide

1. Overview

1.1 Purpose

Define interface design standards between systems within the insurance company (core system, channel platforms, fund platform, data middle platform, etc.), ensuring understandability, consistency, robustness, and maintainability.

1.2 Scope

This standard applies to all synchronous/asynchronous interface designs across systems within the Technology Development Department, including RESTful APIs, gRPC, message queues, and other interaction patterns.

2. Interface Design Principles

2.1 General Principles

1. **Contract First** — Define the interface contract before implementing the code
2. **Backward Compatibility First** — Interfaces are backward-compatible by default; breaking changes must go through the change process
3. **Idempotency** — Write operations (especially those involving financial accounting) must support idempotency
4. **Stateless** — Server should not depend on client session state
5. **Single Responsibility** — One interface does one thing

2.2 Protocol Selection

Scenario	Recommended Protocol	Rationale
Frontend → Backend query	RESTful / GraphQL	Browser-friendly, easy caching
System-to-system sync call	RESTful / gRPC	Structured, type-safe
Async notification/event-driven	Message Queue (RocketMQ/Kafka)	Decoupling, peak shaving
Batch file transfer	SFTP / S3	Large files, resume support
Real-time data subscription	CDC (Debezium/Canal)	Low latency, transactional consistency

3. RESTful API Standards

3.1 URL Naming Convention

`/ {domain} / {context} / {version} / {resource} [ / {resource_id} ] [ ?query_params ]`

Examples:

```
/v1/underwriting/{proposal_no}/decision
/v1/claims/{claim_no}/documents
/v2/policy/{policy_no}/endorsements
```

Naming Rules:

- All lowercase
- Word separators use hyphens -
- Do not use underscores _
- Do not use file extensions (.json, .xml)
- Use path variables for IDs, do not embed them in path segments

3.2 HTTP Method Usage

Method	Purpose	Idempotent	Safe
GET	Query resource	✓	✓
POST	Create resource / trigger operation	✗	✗
PUT	Full resource update	✓	✗
PATCH	Partial resource update	✗	✗
DELETE	Delete resource	✓	✗

3.3 Request & Response Standards**Request Headers:**

Content-Type: application/json
 Accept: application/json
 X-Request-ID: <UUID> // Full trace ID
 X-Channel: <channel_code> // Channel identifier
 X-User-ID: <user_identifier> // Operator identifier
 Idempotency-Key: <UUID> // Idempotency key (mandatory for write operations)

Unified Response Structure:

```

{
  "code": 200,
  "message": "success",
  "data": { },
  "requestId": "a1b2c3d4-...",
  "timestamp": 1718000000000
}

```

Paginated Response:

```

{
  "code": 200,
  "data": {
    "items": [],
    "pagination": {
      "page": 1,
      "pageSize": 20,
      "totalItems": 156,
      "totalPages": 8
    }
  }
}

```

3.4 Error Code Standards

Error Code Range	Meaning
200	Success
400 - 4xx	Client error
500 - 5xx	Server error

Business Error Code Structure: {HTTP Status}{3-digit business code}

Examples:

- 400001 — Parameter validation failure
- 400002 — Policy status does not permit operation
- 400003 — Request already processed (idempotency rejection)
- 500001 — Core system service unavailable

3.5 API Version Management

- Version identifier in URL path: /v1/..., /v2/...
- Backward-compatible changes (new fields, new optional parameters) do not require version upgrades
- Incompatible changes must increment major version
- Old versions maintained for at least 6 months, with 3 months advance notice to consumers

4. gRPC Standards

4.1 .proto File Standards

```
syntax = "proto3";
package insurance.underwriting.v1;
service UnderwritingService {
  rpc Evaluate(UnderwritingRequest) returns (UnderwritingResponse);
  rpc QueryDecision(QueryRequest) returns (DecisionResult);
}
message UnderwritingRequest {
  string proposal_no = 1;
  string product_code = 2;
}
```

4.2 Conventions

- Package name format: {company}.{domain}.{service}.v{version}
- Field numbers start at 1, reserve 1-15 for high-frequency fields
- Use google.protobuf Well-Known Types
- Enum values start with uppercase, use ENUM_NAME_UNSPECIFIED = 0

5. Message Queue Standards

5.1 Message Structure

```
{
  "messageId": "UUID",
  "producer": "underwriting-service",
  "timestamp": 1718000000000,
  "type": "POLICY_ISSUED",
  "version": "1.0",
  "traceId": "UUID",
  "payload": { }
```

```
}
```

5.2 Topic Naming

```
{domain}.{event_type}.{version}
```

Examples:

```
underwriting.proposal.submitted.v1
```

```
policy.policy.issued.v1
```

```
claims.claim.settled.v1
```

5.3 Message Reliability

- Producer: Send acknowledgment (ACK) mechanism, retry 3 times, route to DLQ on exhaustion
- Consumer: Manual ACK on successful consumption, retry on failure (max 16 times), route to DLQ on exhaustion
- Critical messages (accounting-related) must use transactional messaging

6. Interface Documentation Requirements

6.1 Mandatory Content

1. **Overview** — Purpose, involved systems, business trigger scenario
2. **Request Description** — HTTP method, URL, Headers, request body example
3. **Response Description** — Normal response, error response, error code list
4. **Field Definition Table** — Field name, type, length, required, description, enum values

6.2 Documentation Tools

- OpenAPI 3.0 (Swagger) for RESTful APIs
- Protobuf files for gRPC
- AsyncAPI for event-driven interfaces
- All interface documents centrally managed at Interface Management Platform

6.3 Field Definition Table Template

Field Name	Type	Length	Required	Description	Enum Values / Example
policy_no	String	20	Y	Policy number	P2024000001
premium	Decimal	18,2	Y	Premium amount	1000.00
status	String	10	Y	Policy status	ACTIVE/LAPSE/SURRENDER

7. Interface Review

7.1 Review Checklist

- ☐ URL naming follows conventions
- ☐ Request/response format is unified
- ☐ Error codes cover all exception scenarios
- ☐ Idempotency solution designed
- ☐ Security controls (authentication, tamper-proof, replay prevention)
- ☐ Rate-limiting solution defined
- ☐ API documentation managed via tools, not Word
- ☐ Field change compatibility assessed

7.2 Security Requirements

- All interfaces must use OAuth2.0 / JWT authentication
 - Sensitive data (ID numbers, phone numbers, bank cards) must be encrypted during transmission
 - Core interfaces must enable signature verification
 - Replay protection: nonce + timestamp mechanism
 - Rate limiting: token bucket per interface × channel × consumer
-

8. Appendix

8.1 Reference Documents

- Google API Design Guide
- OpenAPI 3.0 Specification
- gRPC Best Practices
- *Core System Data Model Design Standard*

8.2 Templates

- API documentation template: [Internal link]
- OpenAPI example file: [Internal link]
- gRPC proto file example: [Internal link]